

7) AI Orchestration, Extraction and Checks

Last updated by | Mennell, Jacob | 19 May 2026 at 14:02 BST

AI Orchestration, Extraction, And Checks

Purpose

This page explains how the main worker coordinates AI-assisted extraction and checking flows, including execution routing, retrieval-backed task execution, QA, and retry behavior.

Scope

The solution uses two closely related orchestration patterns:

- extraction workflows, which focus on deriving structured outputs from claim evidence
- checks workflows, which evaluate or validate claim-related decisions using bounded and observable execution loops

Both patterns are graph-driven, but they preserve a clear separation between macro routing and micro execution behavior.

Core Design Principle

The orchestration architecture is deliberately split between graph-level and node-level ownership.

Graph ownership

The graph controls macro flow, including:

- which major path is entered
- whether a retry happens
- whether execution ends or loops back

Node ownership

Nodes control the mechanics of execution, including:

- retrieval assembly
- prompt or agent construction
- model invocation
- fail-soft behaviour
- QA input shaping
- bounded retry handling inside a local execution slice

This keeps routing policy and execution policy separate.

Extraction Orchestration Model

The extraction architecture uses three major routing dimensions:

1. retrieval strategy

This selects the broad extraction pattern, such as normal, progressive-stop, progressive-fanin, or web-search-assisted retrieval.

2. execution mode

This chooses how a single execution pass behaves, typically deterministic or agentic.

3. QA lifecycle

This validates outputs after execution and can trigger bounded retries.

The result is a flexible but explicit runtime model that reuses common execution functions while still supporting specialized orchestration behaviour.

Extraction Modes

Normal mode

This is the standard retrieval-backed execution path. Evidence is retrieved globally for the task, then a deterministic or agentic pass executes over that evidence.

Progressive-stop

This mode performs ordered per-document scanning with early stopping. The system attempts extraction against one document at a time and stops on the first accepted result.

Important aspects of this mode include:

- primary versus context evidence separation
- optional rolling context carried from prior attempts
- optional same-document deterministic fallback after an agentic miss
- a terminal fail-safe path when no attempt is accepted

Progressive-fanin

This mode uses a map-reduce style approach. Documents are processed individually, successful outputs are accumulated, and a reduce phase combines them into a final answer.

Web-search retriever mode

This is a specialised path for workflows that require tool-enabled web search behaviour rather than only claim-document retrieval.

Checks Orchestration Model

The checks architecture follows a simpler but still explicit loop:

1. run the check
2. evaluate the result in QA
3. retry only when policy allows it
4. end with either an accepted result or a safe fail-soft output

The checks path is designed to keep retries bounded and deterministic. When recoverable failures persist, the system can emit a safe `stuck` decision rather than failing the entire workflow silently or unpredictably.

QA And Retry Behaviour

QA is treated as a terminal guardrail rather than a side effect of execution. This is an important architectural choice because it keeps validation policy consistent even when the execution mode changes.

Across both extraction and checks, the repository shows several retry layers that are intentionally bounded:

- transient or runner-level retry for infrastructure issues
- node-level constrained retry for recoverable local failures
- graph-level retry loops controlled by QA policy

For extraction, QA can also reroute retries to a more stable execution path. For example, progressive-stop retries are forced back through deterministic execution while preserving the broader retrieval strategy.

Fail-Soft Behaviour

The orchestration flow is designed to preserve output and telemetry contracts even when ideal execution does not succeed.

Examples include:

- terminal fail-safe behaviour in progressive-stop extraction
- safe `stuck` outputs for checks after bounded recoverable failures

This reduces the risk of uncontrolled workflow failure and gives operators clearer insight into how the system behaved.

Orchestration Flow Diagram



flowchart TD

```

START[Workflow Start] --> MODE[Select Retrieval Strategy]
MODE --> NORMAL[Normal Execution]
MODE --> PSTOP[Progressive Stop]
MODE --> PFAN[Progressive Fan-In]
MODE --> WEB[Web Search Retriever]
NORMAL --> QA[QA Evaluation]
PSTOP --> QA
PFAN --> QA
WEB --> QA
QA -->|Pass| END[Finish]
QA -->|Retry Allowed| RETRY[Retry Preparation]
RETRY --> DET[Deterministic or Constrained Retry Path]
DET --> QA
QA -->|Retry Exhausted or Fail-Soft| END

```

Why This Architecture Fits The Solution

This orchestration model suits a claims AI platform because it separates evidence retrieval, reasoning style, and validation policy instead of collapsing them into one opaque model call. That makes the system easier to tune, observe, and stabilize over time.

It also allows the platform to use different execution strategies depending on the problem:

- early-stop behaviour when one strong source is expected to answer the question
- fan-in behaviour when multiple documents must be combined
- deterministic fallback when agentic behaviour is too costly or unstable for a retry

Operational Notes

- QA pass rates, retry counts, acceptance depth, and fail-soft rates are meaningful operational signals for tuning.
- Retrieval quality and orchestration quality are tightly linked; weak evidence quality will often surface as QA retries or unstable task outputs.
- Progressive modes are more behaviourally complex than normal mode and deserve dedicated observability.